

STUDENT NAME : Nurdaulet Serketay
STUDENT ID: 230103358
HOST CPU / SOC MODEL : Apple M2
OPERATING SYSTEM & KERNEL: macOS 25C56

TASK 1: Host Silicon Audit & Flynn’s Taxonomy Mapping

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	Apple M2 (ARM64)	Base & Max Boost Clock	2.42 GHz / 3.49 GHz (Dynamic DVFS)
Physical Hardware Cores	8 (4 Performance + 4 Efficiency)	Logical Cores (SMT/Threads)	8
L1 Data / Instruction Cache	P-cores: L1d: 128 KB L1i: 192 KB E-cores: L1d: 64 KB L1i: 128 KB	L2 Cache (Per Core / Total)	20 MB Total (16 MB P-cluster + 4 MB E-cluster)
L3 Shared Last-Level Cache	N/A (8 MB System Level Cache)	SIMD Vector Extensions	ARM NEON (AdvSIMD)

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy

- SISD (Single Instruction, Single Data): Occurs during single-threaded, sequential execution on a single core of the Apple M2. A single pipeline processes one instruction stream operating on one scalar data stream at a time (e.g., single-threaded C compilation via clang, standard Python execution, or sequential file I/O).

- SIMD (Single Instruction, Multiple Data): Executed using the ARM NEON (AdvSIMD) vector engine with 128-bit vector registers. A single vector instruction processes multiple data elements simultaneously across vector registers (e.g., dense matrix multiplication, signal processing, or accelerated math routines in Apple's Accelerate framework and NumPy).
- MIMD (Multiple Instruction, Multiple Data): The primary execution mode of the macOS multi-threading subsystem across all 8 physical cores (4 Performance + 4 Efficiency). Multiple independent CPU cores execute separate instruction streams concurrently on distinct memory addresses (e.g., the OS thread scheduler running background system services on E-cores while allocating parallel benchmark worker processes across P-cores).

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN=T1/TN	Efficiency EN=SN/N
N = 1(Baseline)	19.567	20.892	20.363	20.274	1.00x	100.0%
N = 2	12.437	12.142	12.007	12.195	1.66x	83.1%
N = 4	7.082	12.409	7.986	9.159	2.21x	55.3%
N = 8	5.091	5.394	5.780	5.422	3.74x	46.7%
N = 16	4.640	5.398	4.384	4.807	4.22x	26.4%
N = 32	4.577	4.557	4.492	4.542	4.46x	13.9%

Q2.1: Scaling Saturation & Hardware Contention Analysis

Empirical results demonstrate severe sub-linear scaling, with parallel efficiency dropping from 100% at $N=1$ down to 46.7% at $N=8$ and collapsing to 13.9% at $N=32$. Speedup nearly plateaus past $N=8$ (gaining only from $3.74\times$ to $4.46\times$ despite quadrupling thread count). This inflection point is driven by three main hardware and runtime factors:

- **Asymmetric Hybrid Core Topology (P-cores vs. E-cores):** The Apple M2 features 4 Performance cores (~ 3.49 GHz) and 4 Efficiency cores (~ 2.42 GHz). Initial workloads ($N \leq 4$) run on high-throughput P-cores. When scaling to $N=8$, work offloads onto slower E-cores, causing load imbalance where faster threads stall waiting for slower threads to complete their assigned work chunks.
- **Physical Core Limit & Lack of SMT:** The M2 host has 8 physical cores and 0 SMT/Hyperthreading extensions ($8\text{ physical} = 8\text{ logical threads}$). Spawning $N=16$ or $N=32$ processes oversubscribes available physical execution pipelines, forcing the macOS thread scheduler to introduce context-switching overhead and thread time-slicing without providing additional hardware compute units.
- **Shared Cache & Memory Contention:** As process counts scale beyond 8, concurrent memory accesses across worker processes cause heavy contention on the shared 16 MB L2 cache (P-cluster) and system memory bus, leading to CPU pipeline stalls.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap

Run	Measured Output	Error (107-Act)	Run	Measured Output	Error (107-Act)
#1	2,900,392	7,099,608	#6	2,676,247	7,323,753
#2	2,715,959	7,284,041	#7	2,080,199	7,919,801
#3	2,590,293	7,409,707	#8	2,789,901	7,210,099
#4	2,852,459	7,147,541	#9	2,260,607	7,739,393

#5	2,757,654	7,242,346	#10	2,622,969	7,377,031
----	-----------	-----------	-----	-----------	-----------

Q3.1: Read-Modify-Write (RMW) Window

- At the ARM64 machine instruction level, counter++ is non-atomic and executes in three distinct steps: Load (LDR from cache/RAM into a register), Add (ADD 1 to the register value), and Store (STR register value back to cache/RAM).
- When multiple threads execute across cores simultaneously without locking, Core B loads the shared variable before Core A stores its modified value back.
- Core B increments a stale value, overwriting Core A's stored update and discarding increments (causing an error of ~70%–79%).

Q3.2: The Synchronization Penalty

- Measured Execution Time: Unlocked = 6.82 ms vs. Locked = 228.15 ms.
- Serialization: Mutex acquisition converts parallel execution into single-threaded sequential execution, eliminating core concurrency.
- Hardware Cache Line Bouncing: Synchronizing access forces atomic operations (LDREX/STREX or memory barriers) that send invalidation signals across core L1/L2 caches, forcing cores to constantly invalidate and reload cache lines.
- Kernel & Contention Overhead: Lock acquisition and release across 10,000,000 loop iterations incurs repeated lock contention, OS kernel context switching, and thread sleep/wake overhead.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

task 2 measurements (T1=20.274 s, T2=12.195 s):

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon

1. Parallel Fraction Derivation (p)

- Formula: $p = 2 * (T1 - T2) / T1$

- $T1 = 20.274 \text{ s}$
- $T2 = 12.195 \text{ s}$
- Calculated $p = 2 * (20.274 - 12.195) / 20.274 = 0.797$ (or 79.7 percent)

2. Amdahl Theoretical Speedup Ceiling

- Formula: $S_{max} = 1 / (1 - p)$
- Sequential fraction $(1 - p) = 1 - 0.797 = 0.203$ (or 20.3 percent)
- Max Theoretical Speedup (N approaches infinity) $= 1 / 0.203 = 4.93x$

3. Amdahl 64-Core Projection

- Formula: $S_{64} = 1 / ((1 - p) + (p / 64))$
- Projected Speedup on 64 Cores $= 1 / (0.203 + (0.797 / 64)) = 1 / (0.203 + 0.01245) = 4.64x$

4. Gustafson Scaled Speedup (Weak Scaling)

- Formula: $S_{Gustafson} = (1 - p) + (p * 64)$
- Projected Scaled Speedup (64x Problem) $= 0.203 + (0.797 * 64) = 0.203 + 51.008 = 51.21x$

Q4.1: Strong vs. Weak Scaling Analysis:

Amdahl's Law models strong scaling, which keeps the total problem size fixed as processor count grows. As the core count approaches infinity, the time spent on parallel execution approaches zero, leaving total performance strictly limited by the non-parallelizable sequential portion of the code. In this workload, the fixed sequential fraction of 0.203 creates an absolute performance ceiling of 4.93x speedup regardless of how many cores are added.

Gustafson's Law models weak scaling, which assumes the problem size grows proportionally alongside the increase in available processor cores so that work per core stays constant. Because total parallel workload scales up by 64 times on a 64-core system, the unparallelizable sequential fraction becomes a tiny fraction of the total execution time rather

than a dominant bottleneck. This allows Gustafson's model to predict a near-linear speedup of 51.21x for the expanded workload.